



Software Design Document

Color Clash - A Two-Player Fighting Game

TOBB ETU – BIL 482

March 2026

Team Members

Mehmet Umur ÖZÜ

Mehmet Yasin TOSUN

Mustafa GÖZÜTOK

Table of Contents

1. System Overview
2. System Context
3. Key Features and Functionality
4. Assumptions and Dependencies
5. Architectural Design
 - 5.1 System Architecture Diagram
 - 5.2 Architectural Patterns and Styles
 - 5.3 Rationale for Architectural Decisions
6. Component Design
 - 6.1 Subsystems and Modules
 - 6.2 Responsibilities of Each Component
 - 6.3 Component Diagrams
 - 6.3.1 UML Class Diagram A – Core Architecture
 - 6.3.2 UML Class Diagram B – Gameplay & Design Patterns
 - 6.3.3 Class / Interface Responsibility Summary
 - 6.4 Interfaces Between Components
 - 6.4.1 Core Engine Interfaces
 - 6.4.2 Gameplay / Combat Interfaces
 - 6.4.3 UI / Observer Interfaces
 - 6.4.4 Performance / Pooling Interfaces
7. Data Design
 - 7.1 Data Model
 - 7.2 Data Flow Diagrams
8. Design Patterns
 - 8.1 Applied Design Patterns
 - 8.2 Context and Justification
9. Implementation Notes
10. User Interface Design
11. Performance Considerations
12. Error Handling and Logging
13. Deployment and Installation Design

14. Change Log

15. Future Work / Open Issues

16. Task Assignment Matrix

1. System Overview

Color Clash is a web-based 2D fighting game developed using TypeScript and HTML5 Canvas. Unlike traditional high-violence fighting games, this project emphasizes a vibrant, child-friendly aesthetic and strategic depth. It serves as a technical case study for implementing advanced software design patterns in a real-time game engine environment.

2. System Context

The application is a standalone, client-side web application.

- **Platform:** Desktop/Laptop web browsers (Chrome, Firefox, Safari).
- **Technology Stack:** TypeScript (Logic), HTML5 Canvas (Rendering), Vite (Build System).
- **External Interfaces:** Browser DOM for UI and the Keyboard API for user input.

3. Key Features and Functionality

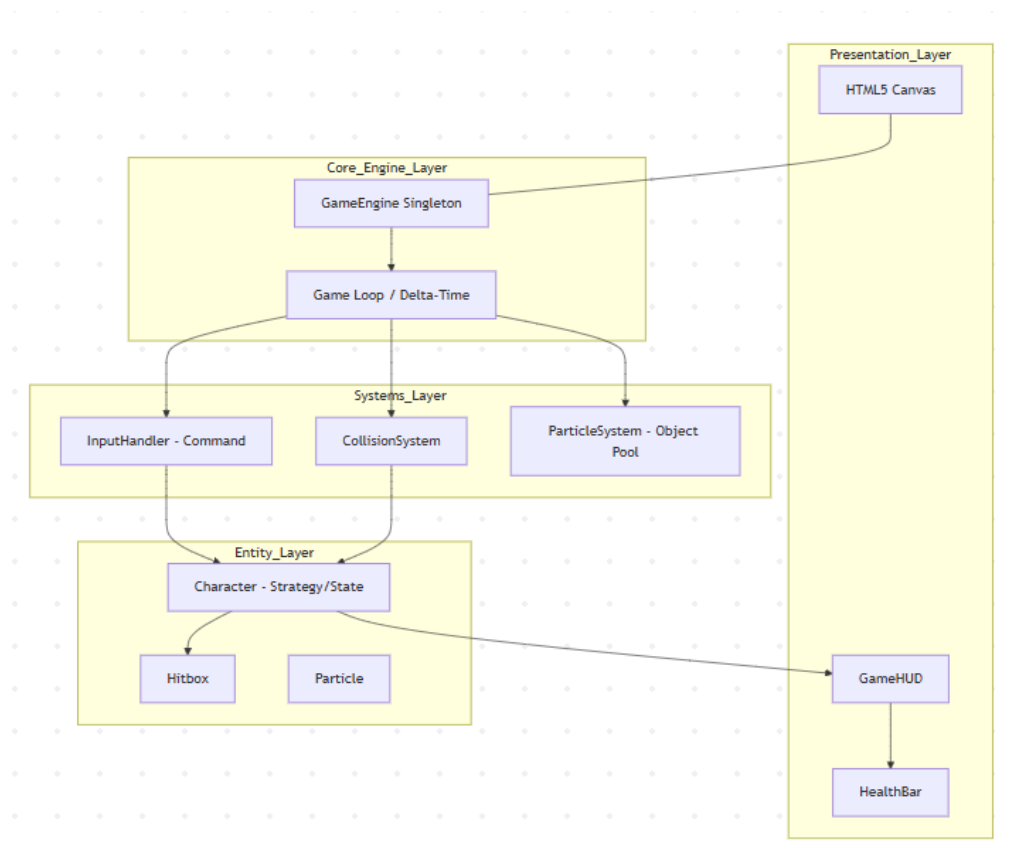
- **UC1 - Start Match or Tournament:** Initialization of the singleton game engine and setup of the core game state. Players can choose to start a classic 1v1 duel or initiate a "Tournament Mode" for up to 8 competitors. The system handles character instantiation dynamically via a Factory and sets up input listeners based on active matches.
- **UC2 - Execute Combat Actions:** Real-time character control including movement (left, right, jump), accurate AABB collision resolution between players, and hitbox-based combat processing (attacks, hit stun, and knockback).
- **UC3 - Dynamic Elemental Strategy (6 Modes):** Implementation of the Strategy pattern allowing characters to toggle between six distinct elemental modes at runtime, affecting combat attributes, visuals, and movement physics dynamically. The available modes are:
 - Fire: Aggressive form with High Damage multiplier.
 - Water: Defensive form focusing on High Defense and fluid combat.
 - Earth: Armored form providing the highest defense at the cost of mobility.
 - Wind: Aerial form that reduces gravity and enables flight capabilities.
 - Light: Ranged form specializing in massive attack range multipliers and beam visuals.
 - Dark: Absorption form that converts a portion of incoming damage into bonus attack power.
- **UC4 - Manage Tournament Bracket:** Automated handling of tournament brackets. The system pairs competitors, tracks match results, advances winners to subsequent rounds (including automatic byes if necessary), displays round progress animations, and finally reveals a championship podium for the top 3 players.

4. Assumptions and Dependencies

- **Assumptions:** Players interact via a single shared keyboard for local multiplayer.
- **Dependencies:** Requires Node.js for development, TypeScript for compilation, and Vite for the development server and bundling.

5.1 System Architecture Diagram

The system follows a Layered Architecture to decouple game logic from the rendering hardware.



5.2 Architectural Patterns and Styles

- **Singleton Pattern:** Applied to the GameEngine to ensure a single authoritative source for the game loop and state management.
- **Layered Style:** Separates core combat physics from the UI/Rendering logic, facilitating independent testing.

5.3 Rationale for Architectural Decisions

The Singleton ensures that multiple engine instances do not compete for the CPU or Canvas context. The Layered Architecture allows for future-proofing; for instance, the rendering layer could be swapped for WebGL without modifying the core physics logic.

6. Component Design

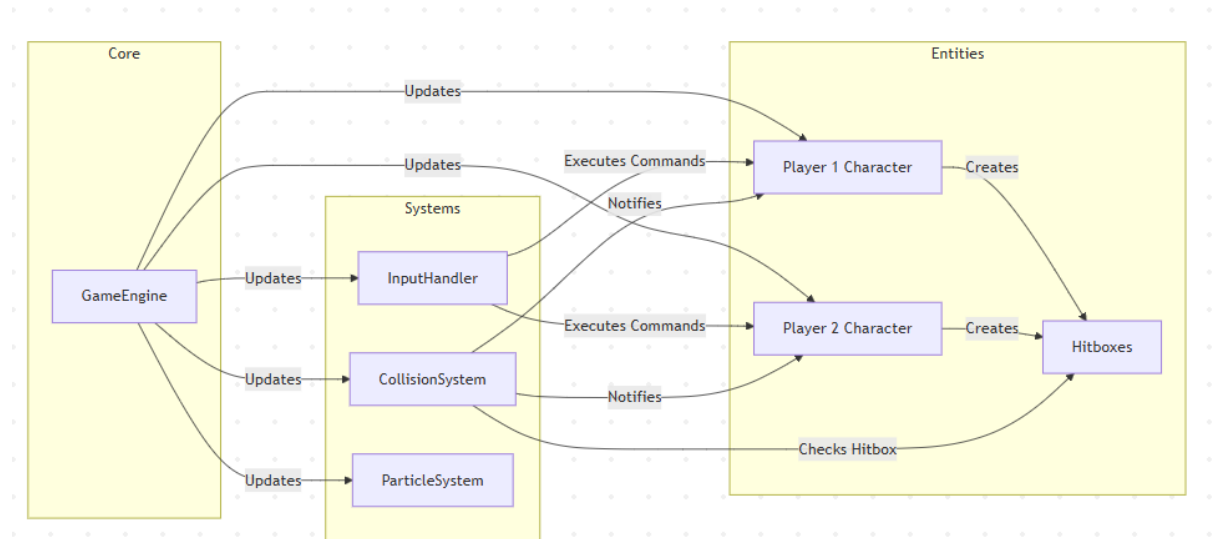
6.1 Subsystems and Modules

- **GameEngine:** Orchestrates the initialization and the main requestAnimationFrame loop.
- **InputHandler:** Maps raw browser keyboard events to specific game commands.
- **CollisionSystem:** Handles AABB (Axis-Aligned Bounding Box) logic to prevent overlaps and detect hits.
- **CharacterFactory:** Centralized module for creating player entities with specific configurations.

6.2 Responsibilities of Each Component

- **Character:** Manages individual health, position, velocity, and current state.
- **Physics Engine:** Applies gravity and friction constants to active entities.

6.3 Component Diagrams



The diagram below presents the structural backbone of Color Clash. GameEngine (Singleton) acts as the central orchestrator, owning InputHandler, ParticleSystem, and GameHUD as subsystems. Character extends Subject<T>, enabling it to publish HealthChangeEvent and ModeChangeEvent notifications. Both Hitbox and Particle implement IPoolable for pool-compatibility.

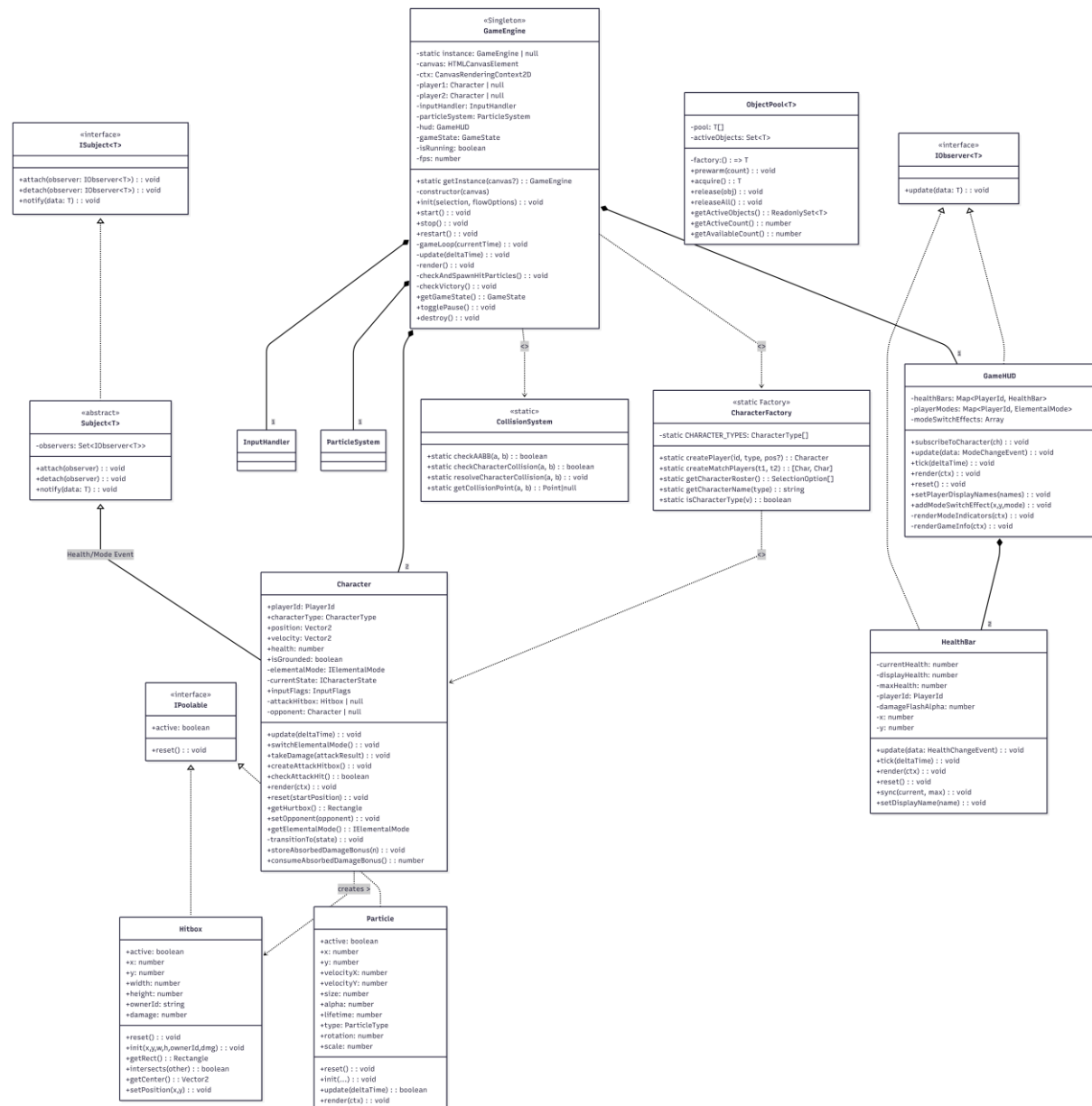


Diagram B isolates the four design patterns. Command pattern (ICommand / MoveCommand / AttackCommand / SwitchModeCommand) decouples keyboard events from character actions. State pattern (ICharacterState / IdleState / MoveState / AttackState / HitState) governs per-frame behaviour. Strategy pattern (IElementalMode / FireModeStrategy / WaterModeStrategy) enables hot-swapping of combat logic. Observer pattern (Subject<T> / HealthBar / GameHUD) propagates game events to UI.



Class / Interface	Pattern Role	Responsibility	Key Public API
GameEngine	Singleton	Central game-loop orchestrator; owns all subsystems	getInstance(), init(), start(), stop(), restart(), togglePause()
Character	Context (State+Strategy)+Subject	Main player entity; delegates behaviour to state/strategy; publishes events	update(), render(), takeDamage(), switchElementalMode(), checkAttackHit(), reset()
Hitbox	Value Object/IPoolable	AABB collision box for attack detection; poolable	init(), intersects(), getRect(), reset()
Particle	IPoolable	Single visual-effect particle with physics and typed rendering	init(), update(), render(), reset()
CharacterFactory	Factory Method	Constructs and wires two Character instances	createPlayer(), createMatchPlayers()

InputHandler	Command Dispatcher	Maps KeyboardEvents to ICommand objects and executes them	registerPlayer(), destroy()
CollisionSystem	Utility (static)	Stateless AABB detection and resolution	checkAABB(), resolveCharacterCollision(), getCollisionPoint()
ParticleSystem	Pool Consumer	Spawns and manages particles via ObjectPool<Particle>	spawnHitEffect(), update(), render(), clear()
GameHUD	IObserver<ModeChangeEvent>	Composite UI; owns HealthBars; observes mode-change events	subscribeToCharacter(), tick(), render(), reset()
HealthBar	IObserver<HealthChangeEvent>	Animated health bar; receives health events from Character	update(), tick(), render(), reset()
ICommand	Command Interface	Decouples keyboard input from character actions	execute(character)
MoveCommand	Concrete Command	Sets directional inputFlags on the character receiver	execute(character)
AttackCommand	Concrete Command	Sets inputFlags.attack on the character receiver	execute(character)
SwitchModeCommand	Concrete Command	Calls character.switchElementalMode()	execute(character)
ICharacterState	State Interface	Contract for all character behaviour states	enter(), update(), exit(), canTransition()
IdleState	Concrete State	Default resting state; transitions to Move or Attack on input	All ICharacterState methods
MoveState	Concrete State	Applies velocity from inputFlags; handles jumping	All ICharacterState methods
AttackState	Concrete State	Creates hitbox, runs hit detection, times attack window	All ICharacterState methods
HitState	Concrete State	Applies knockback decay and stun timer; blocks inputs	All ICharacterState methods
IElementalMode	Strategy Interface	Contract for elemental combat strategies	attack(), defend(), getVisualConfig(), getParticleType()
FireModeStrategy	Concrete Strategy	High damage x1.5, low defence x0.7, fast attack x1.2	All IElementalMode methods
WaterModeStrategy	Concrete Strategy	Moderate damage x0.8, high defence x1.5, slower attack x0.9	All IElementalMode methods
Subject<T>	Abstract Subject	Generic observable; manages Set<IObserver<T>>	attach(), detach(), notify()
IObserver<T>	Observer Interface	Single-method callback for event delivery	update(data: T)
ObjectPool<T>	Object Pool	Pre-allocates reusable poolable objects; prevents GC spikes	acquire(), release(), releaseAll(), getActiveObjects()
GameConfig	Constants Module	Centralised numeric and structural game constants	Exported const values

6.4 Interfaces Between Components

The following tables formally document the major component interaction points. Each entry lists the calling component, receiving component, data transferred, return value, and observable side effects.

6.4.1 Core Engine Interfaces

Interaction	Caller	Callee	Inputs	Returns	Side Effects
GameEngine.getInstance(canvas)	main.ts::initGame()	GameEngine	HTMLCanvasElement (first call)	GameEngine singleton	Constructs engine, subsystems, sets canvas size
GameEngine.init()	main.ts::initGame()	GameEngine	--	void	Creates players via CharacterFactory, registers w/ InputHandler, subscribes HUD observers
GameEngine.start()	main.ts::initGame()	GameEngine	--	void	Sets isRunning=true, schedules requestAnimationFrame
GameEngine.update(deltaTime)	GameEngine::gameLoop()	GameEngine internal	deltaTime: number (capped 0.1 s)	void	Updates players, resolves collisions, ticks particles & HUD, spawns hit particles, checks win
GameEngine.render()	GameEngine::gameLoop()	GameEngine internal	--	void	Draws: background -> particles -> characters -> HUD -> victory overlay
checkAndSpawnHitParticles()	GameEngine::update()	CollisionSystem, ParticleSystem	--	void	Checks active hitboxes via getCollisionPoint(); calls spawnHitEffect() on overlap
GameEngine.restart()	showVictory() / keydown R	GameEngine	--	void	Resets player positions, clears particles, resets HUD and gameState
GameEngine.togglePause()	main.ts keydown P	GameEngine	--	void	Toggles isPaused; paused game skips update() but still calls render()

6.4.2 Gameplay / Combat Interfaces

Interaction	Caller	Callee	Inputs	Returns	Side Effects
InputHandler::handleKeyDown()	Browser keydown event	InputHandler	Keyboard Event	void	Calls mapKeyToCommand() then

					command.execute(character)
InputHandler::mapKeyToCommand()	InputHandler::handleKey Down/Up	InputHandler internal	keyCode, KeyBindings, pressed, PlayerId	ICommand null	Constructs MoveCommand, AttackCommand, or SwitchModeCommand
MoveCommand.execute(character)	InputHandler	Character	Character reference	void	Sets character.inputFlags.{l r u d}; recalculates .moving
AttackCommand.execute(character)	InputHandler	Character	Character reference	void	Sets character.inputFlags.ata ck = pressed
SwitchModeCommand.execute(character)	InputHandler	Character	Character reference	void	Calls character.switchElement alMode(); no-op if attacking/stunned
Character.update(deltaTime)	GameEngine::update()	Character -> ICharacterState	deltaTime: number	void	Ticks cooldown; delegates to currentState.update(); evaluates canTransition()
ICharacterState.update(ch, dt)	Character.update()	Concrete State	Character, deltaTime	void	Mutates velocity/position; calls applyGravity, checkGroundCollision, checkBoundaries
ICharacterState.canTransition(ch)	Character.update()	Concrete State	Character	ICharacterState null	Reads inputFlags and timers; returns next state or null
AttackState.enter(character)	Character::transitionTo()	AttackState	Character	void	Sets isAttacking=true, calls startAttackCooldown() and createAttackHitbox()
Character.checkAttackHit()	AttackState.update()	Character, IElementalMode	--	boolean	Tests hitbox.intersects(oppone nt.getHurtbox()); on hit calls elementalMode.attack() then opponent.takeDamage()
IElementalMode.attack(att, tgt)	Character::checkAttackHit() ()	FireMode/Water ModeStrategy	Character(att), Character(t gt)	AttackRe sult	Computes damage = BASE_ATTACK_DAM AGE x damageMultiplier; determines knockback direction
IElementalMode.defend(ch, dmg)	Character.takeDamage()	Active strategy	Character, incomingD amage: number	number (reduced dmg)	Returns incomingDamage / defenseMultiplier
Character.takeDamage(result)	Character::checkAttackHit() ()	Character	AttackRes ult {damage,	void	Applies defence, subtracts health, applies knockback, transitions to

			knockback, hitStun}		HitState, notifies HealthChangeEvent
CollisionSystem.resolveCharacterCollision(a,b)	GameEngine::update()	CollisionSystem static	Character, Character	void	Checks AABB overlap; pushes both characters apart by overlapX/2
CollisionSystem.getCollisionPoint(a,b)	GameEngine::checkAndSpawnHitParticles()	CollisionSystem static	Rectangle, Rectangle	{x,y} null	Returns centre of AABB overlap, or null if no intersection
CharacterFactory.createMatchPlayers()	GameEngine.init()	CharacterFactory static	--	[Character, Character]	Creates two Characters with correct positions/bindings; calls setOpponent() on both

6.4.3 UI / Observer Interfaces

Interaction	Caller	Callee	Inputs	Returns	Side Effects
GameHUD.subscribeToCharacter(ch)	GameEngine.init()	GameHUD, Character	Character	void	Calls character.attach(healthBar) and character.attach(this) to register observers
Subject.notify(data)	Character.takeDamage() / switchElementalMode()	Subject<T> -> IObservable<T>*	HealthChangeEvent or ModeChangeEvent	void	Iterates observers Set and calls observer.update(data) on each
HealthBar.update(data)	Subject.notify() (Character)	HealthBar	HealthChangeEvent {playerId, currentHealth, maxHealth, damage}	void	Updates currentHealth; if damage>0 sets damageFlashAlpha=1
GameHUD.update(data)	Subject.notify() (Character)	GameHUD	ModeChangeEvent {playerId, newMode}	void	Updates playerModes map for mode indicator rendering
GameHUD.tick(deltaTime)	GameEngine::update()	GameHUD	deltaTime: number	void	Advances HealthBar animations; fades modeSwitchEffects
HealthBar.tick(deltaTime)	GameHUD.tick()	HealthBar	deltaTime: number	void	Interpolates displayHealth toward currentHealth; decays damageFlashAlpha
GameHUD.render(ctx)	GameEngine::render()	GameHUD	CanvasRenderingContext2D	void	Renders all HealthBars, mode indicators, mode-switch effects, and title text
HealthBar.render(ctx)	GameHUD.render()	HealthBar	CanvasRenderingContext2D	void	Draws gradient health bar, damage flash, border, player label, and numeric HP
GameHUD.reset()	GameEngine.restart()	GameHUD	--	void	Resets all HealthBars, clears modeSwitchEffects,

					resets playerModes to Fire
--	--	--	--	--	----------------------------

6.4.4 Performance / Pooling Interfaces

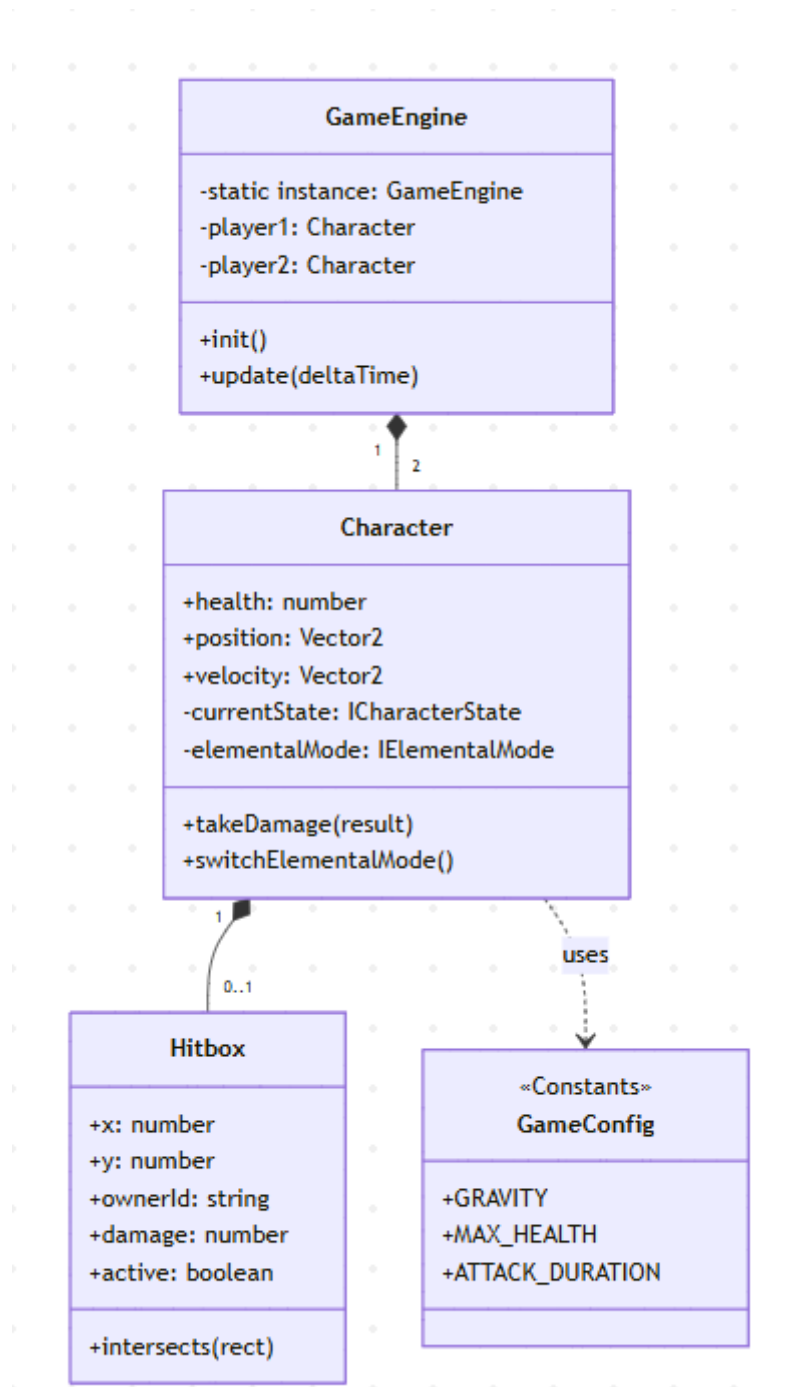
Interaction	Caller	Callee	Inputs	Returns	Side Effects
new ObjectPool<Particle>(factory, size)	new ParticleSystem()	ObjectPool<Particle>	() => Particle factory, PARTICLE_POOL_SIZE=100	ObjectPool instance	Pre-allocates 100 Particle objects via prewarm()
ObjectPool.acquire()	ParticleSystem::spawn()	ObjectPool<Particle>	--	Particle (recycled/new)	Pops from pool[]; sets active=true, calls reset(), adds to activeObjects
Particle.init(x, y, vX, vY, color, type, ...)	ParticleSystem::spawn()	Particle	Position, velocity, visual, lifetime params	void	Populates all particle fields; randomises rotation and rotationSpeed
ObjectPool.release(obj)	ParticleSystem::update()	ObjectPool<Particle>	Particle (expired)	void	Sets active=false, removes from activeObjects, pushes back to pool[]
Particle.update(deltaTime)	ParticleSystem::update()	Particle	deltaTime: number	boolean (alive)	Advances position; applies type-dependent gravity; decays alpha/scale; returns false if expired

ObjectPool.releaseAll()	ParticleSystem.clear()	ObjectPool<Particle>	--	void	Moves all activeObjects back to pool[]; clears activeObjects
ParticleSystem.spawnHitEffect(pos,type)	GameEngine::checkAndSpawnHitParticles()	ParticleSystem	Vector2, ParticleType	void	Calls spawn(pos, type, 15) then spawn(pos, hit, 5) for 20 particles total
ObjectPool.getActiveObjects()	ParticleSystem::update()/render()	ObjectPool<Particle>	--	ReadonlySet<Particle>	Read-only view over activeObjects; no mutation

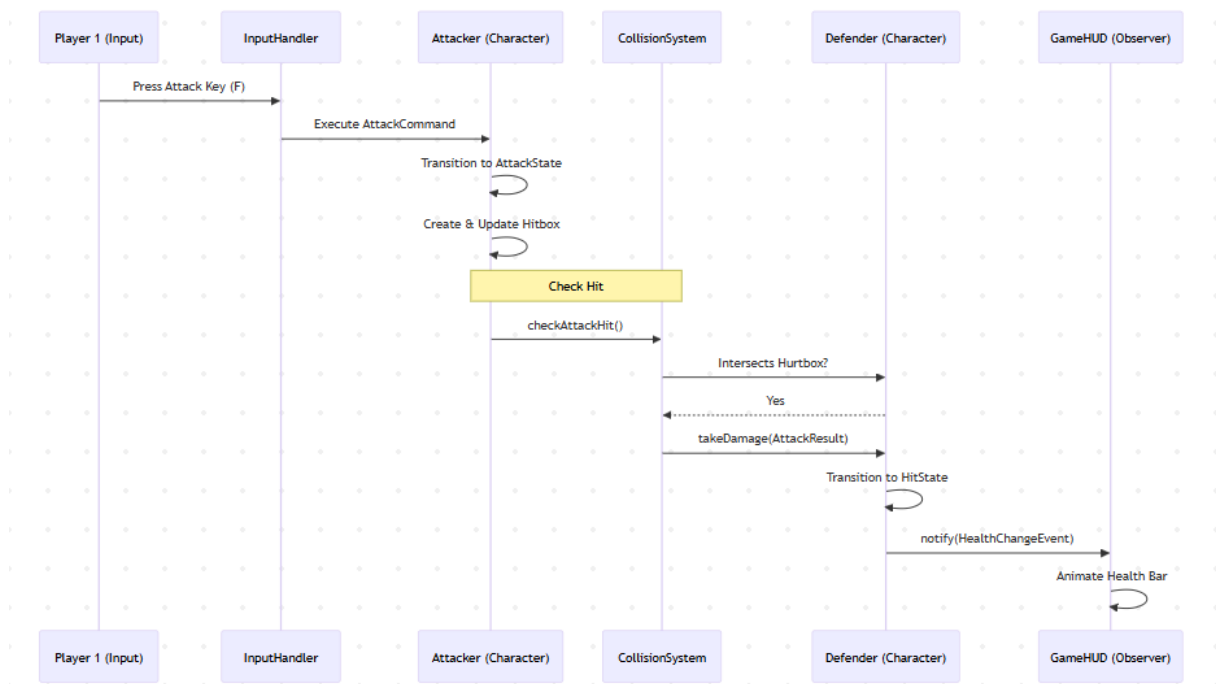
7. Data Design

7.1 Data Model

Data is managed through a central configuration and runtime class instances.



7.2 Data Flow Diagrams



8. Design Patterns

8.1 Applied Design Patterns

Singleton: The GameEngine ensures a single orchestrator for the game loop.

Strategy: IElementalMode allows dynamic switching between all combat attributes.

State: ICharacterState manages transitions between Idle, Move, Attack, and Hit.

Command: ICommand decouples raw keyboard events from the Character logic.

Observer: Decouples the UI (GameHUD) from the core game logic, updating health bars via notifications.

Factory: CharacterFactory centralizes the creation and initialization of player characters.

Object Pool: ParticleSystem reuses particles and hitboxes to prevent performance spikes.

8.2 Context and Justification

The State Pattern is critical to ensure a character cannot perform actions while in a "stunned" (Hit) state. The Strategy Pattern allows for runtime modification of combat stats without altering the core character class.

9. Implementation Notes

Movement uses a velocity-based system with Delta-Time (Δt) integration. This ensures that character speed remains consistent regardless of whether the user has a 60Hz or 144Hz monitor.

10. User Interface Design

The UI consists of a HUD (Heads-Up Display) overlaying the Canvas. It includes:

- **Dynamic Health Bars** (Observer-driven).
- **Mode Indicators.**
- **Game State overlays** (Start/Game Over).

11. Performance Considerations

- **Object Pooling:** Hitboxes and particles are recycled from a pool to prevent "Garbage Collection spikes" during intense combat.
- **Culling:** Entities outside the viewport are not rendered to save GPU cycles.

12. Error Handling and Logging

- **Exception Management:** The GameEngine uses try-catch blocks during initialization to handle failures in canvas context acquisition.
- **Logging:** A custom Logger tracks state transitions and input mapping for debugging purposes.

13. Deployment and Installation Design

- **Environment:** Node.js v18+.
- **Installation:** npm install.
- **Execution:** npm run dev for local development; npm run build for production-ready static files.

14. Change Log

- **v1.0:** Initial core engine and physics implementation.
- **v1.1:** Integrated State, Command, and Strategy patterns.

15. Future Work / Open Issues

- **Audio Engine:** Integration of Web Audio API for sound effects.

16. Task Assignment Matrix

Team Member	Contribution (Classes & Tasks)	Use Case Mapping
Mehmet Umur ÖZÜ	Core & Input Systems: Implemented the GameEngine singleton, InputHandler, IdleState, and the Command pattern infrastructure. Also integrated Strategy pattern for dynamic elemental mode switching.	UC1: Start Game Battle, UC2: Execute Combat Actions, UC3: Switch Elemental Modes,
Mustafa GÖZÜTOK	Entity & Physics: Developed the CharacterFactory, Character class, MoveState, Object Pooling and the CollisionSystem for AABB detection. Contributed to character behavior adaptation for different elemental modes.	UC1: Start Game Battle, UC2: Execute Combat Actions, UC3: Switch Elemental Modes. UC4: Manage Tournament Flow.
Mehmet Yasin TOSUN	Combat & UI: Implemented AttackState, HitState, Hitbox management, main.ts entry point, and authored the Software Design Document (SDD). Developed tournament mode UI and match flow handling.	UC2: Execute Combat Actions, UC3: Switch Elemental Modes, UC4: Tournament Mode Execution.